

SMART NURSERY GUARDIAN

An AI-Assisted Embedded Monitoring System for Infant Care

Final Project Report

Team 6

Name	ID	Role
Mohamed Ibrahim	7131E	Machine Learning / Audio Pipeline / GUI / Telegram
Filopateer Hani	7129E	Hardware / Serial Integration
Tharaa Mohamed	7005E	PCB / Firmware

An intelligent crib-side system that listens to infant cries, classifies the cause using machine learning, and automatically soothes the baby while notifying caregivers.

Table of Contents:

1. Introduction	3
2. Project Objectives	3
3. System Overview & Architecture	4
4. Hardware Design	4
5. Software Design	6
5.1 Streamlit Dashboard (Laptop)	
5.2 Arduino Firmware (Microcontroller)	
5.3 Serial Communication Protocol	
6. Machine Learning Pipeline	8
7. System Workflow (End-to-End)	9
8. Implementation Details & Screenshots	9
9. Testing & Results	11
10. Challenges & Solutions	11
11. Conclusion	12
12. Future Work	12
13. Resources & References	12

1. Introduction:

At 3 a.m., a baby cries. Is she hungry? Tired? In discomfort? The parent stands in darkness guessing. Crying is the infant's only language, and most parents cannot read it fluently. Sleep deprivation, anxiety, and delayed responses are common consequences.

Smart Nursery Guardian is an AI-assisted embedded monitoring system that listens to the cry, uses a trained machine-learning model to identify the cause (hungry, tired, or discomfort), and reacts instantly: rocking the crib, displaying a clear message on an LCD and a modern GUI, sounding a buzzer when needed, and sending a Telegram alert to the caregiver's phone. Nearby sensors continuously monitor motion, light level, air quality, and temperature, controlling lights and cooling automatically.

The system aims to respond as fast and attentively as a caregiver watching the crib — but never sleeps and never misses a signal.

1.1 Why This Project Matters

- Addresses a real, universal problem: sleep loss and stress caused by misinterpreting infant cries.
- Demonstrates a realistic technical stack: signal processing, classical ML, real-time firmware, GUI design, serial IoT communication, and cloud messaging.
- Bridges the gap between academic prototypes and practical home-use systems.

2. Project Objectives:

#	Objective	Success Criterion
1	Classify infant cries into 3 clinically meaningful classes	Macro-F1 ≥ 0.75 on held-out test set
2	Provide real-time visual & audible feedback	LCD + Streamlit GUI update < 1 s after prediction
3	Automatically soothe the baby	Servo rocks continuously while cry is active
4	Detect dangerous gas / smoke	Buzzer + Telegram alert within 2 s of threshold breach
5	Maintain comfortable environment	Fan & RGB LED respond to temperature bands
6	Notify caregiver remotely	Telegram message delivered for every cry & gas event

Cry Class	System Response	Priority
Hungry	Display message on GUI/LCD and play a calming video clip	Normal
Tired	Sound the buzzer immediately and display a warning (urgent pattern)	High
Discomfort	Display message and suggest checking diaper / clothing / position	Normal

3. System Overview & Architecture:

Two programs run on two devices and communicate over a serial (USB) link:

Component	Runs On	Main Responsibilities
Python / Streamlit App	Laptop	Microphone capture, feature extraction, ML inference, GUI dashboard, Telegram alerts, serial command & telemetry
Arduino Firmware	Microcontroller	Read PIR, LDR, thermistor, gas sensor; control servo, buzzer, room LED, RGB status LEDs, cooling fan; send sensor telemetry

3.1 Critical Data Paths

- Audio path (zero latency): Laptop microphone → Python feature extractor → trained model. Audio never crosses the serial link.
- Sensor path: Microcontroller reads sensors and streams values (GAS, TEMP, LIGHT, MOTION) to the laptop every second. Commands (CRY:hungry, RESET ...) travel in the opposite direction.

This asymmetry is intentional: cry detection has the lowest possible latency, while environmental sensors can tolerate a short serial delay.

4. Hardware Design:

4.1 Bill of Materials & Pin Mapping

Component	Arduino Pin	Function
PIR Motion Sensor	D2	Detect baby movement (awake detection)
LDR (Light Sensor)	A0	Room darkness threshold
Thermistor (Temp)	A1	Ambient temperature
MQ-series Gas Sensor	A2	Smoke / gas detection
Room LED (Lamp)	D3	Night light when dark + awake/cry
RGB LED – Red	D9	Hot temperature indicator
RGB LED – Green	D10	Comfortable temperature
RGB LED – Blue	D11	Cold temperature
Cooling Fan (PWM)	D5	Speed controlled by temperature
Fan Direction (IN)	D12	Motor driver enable
Buzzer	D8	Gas alert or tired-cry continuous tone
Servo Motor	D6	Crib rocking (0° ↔ 90° ↔ 180°)
LCD 16×2 (I2C)	0x27	Local status display

4.2 Key Thresholds & Control Logic

Parameter	Value	Action
Darkness (LDR)	< 900	Room considered dark
Gas / Smoke	> 400	Buzzer ON + Telegram + GUI alert
Motion window	8 seconds	Count PIR edges
Awake threshold	≥ 4 motions	isBabyAwake = true
Temp < 25 °C	Blue LED	Fan low (PWM 100)
25–30 °C	Green LED	Fan half (PWM 160)
Temp > 30 °C	Red LED	Fan full (PWM 255)

4.3 Custom PCB Design

A custom 2-layer PCB was designed in Altium Designer (dated 5-9-2026, Team 6). The board integrates the microcontroller, sensor connectors, power regulation, servo and fan drivers, and the Smart Nursery Guardian logo.

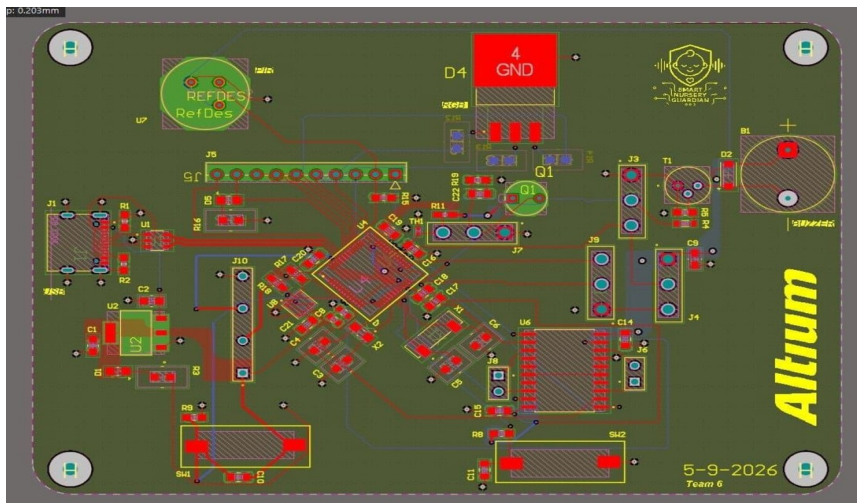


Figure 1 – PCB Layout (Altium Designer)

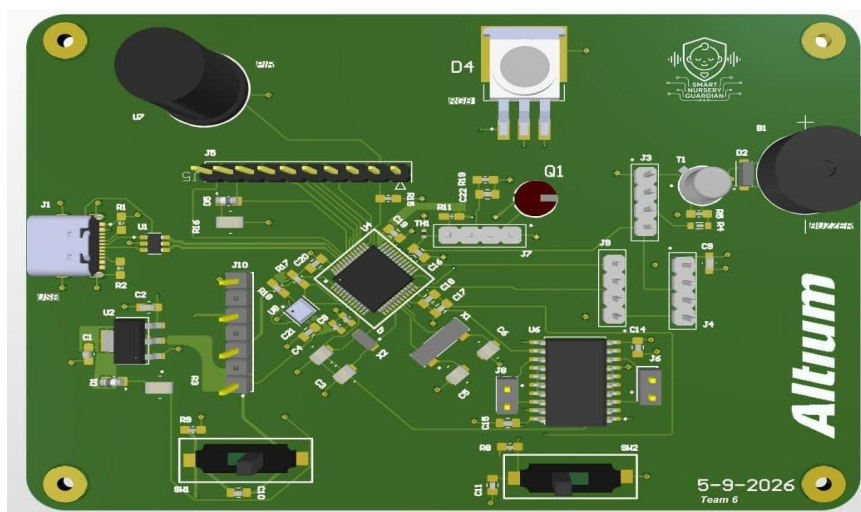


Figure 2 – 3D Render of the Assembled PCB

4.4 Hardware Prototype in Action



Figure 3 – Live LCD showing “Temp:25C Light / Baby: Sleeping” with surrounding sensors and wiring

5. Software Design:

5.1 Streamlit Dashboard (Laptop Side)

The caregiver interface is a responsive Streamlit web application that runs on the laptop. It provides real-time status cards, a serial log, manual audio upload, mock-test buttons, and automatic microphone monitoring.

Main Features

- Live metrics: Temperature / Fan, Motion (Awake/Calm), Gas level, Room Lamp
- Cry classification results with confidence scores
- Auto-mode continuous microphone loop (optional)
- Manual upload of cry audio files (WAV, MP3, ...)
- Telegram notifications for every cry and gas event

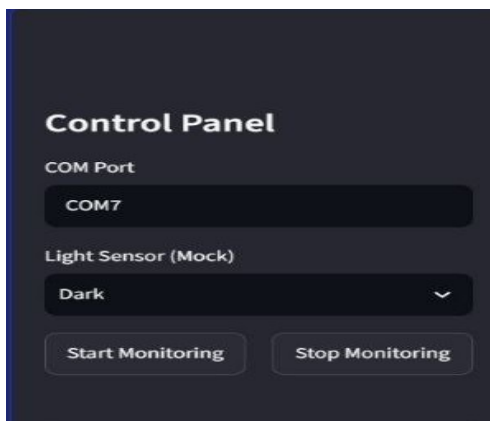


Figure 4 – Control Panel (COM Port & Start/Stop Monitoring)

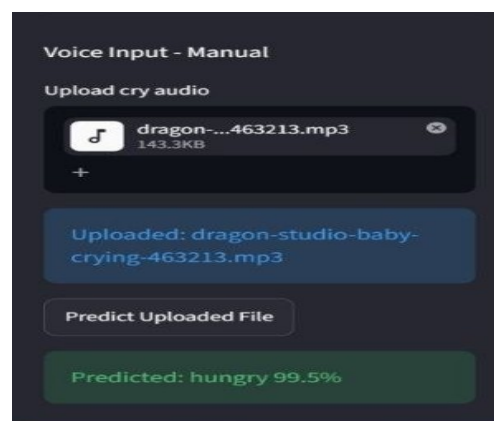


Figure 5 – Manual Voice Upload & Prediction Result (hungry 99.5%)

5.2 Arduino Firmware

The firmware (finalCodeHardware.ino) runs a tight non-blocking loop that:

- Reads serial commands from the laptop (CRY:hungry / tired / discomfort, RESET, ...)
- Samples all sensors
- Updates the 8-second motion window and awake state
- Sends periodic telemetry (GAS, TEMP, LIGHT)
- Controls lighting, temperature actuators, buzzer, and servo rocking
- Refreshes the 16×2 LCD every 300 ms

5.3 Serial Communication Protocol

Direction	Message	Meaning
Laptop → MCU	CRY:hungry	Start hungry response (rock + display)
Laptop → MCU	CRY:tired	Start tired response (rock + continuous buzzer)
Laptop → MCU	CRY:discomfort	Start discomfort response
Laptop → MCU	RESET / CRY:END	Clear cry state, stop buzzer, center servo
MCU → Laptop	GAS:<value>	Raw gas sensor reading
MCU → Laptop	TEMP:<value>	Temperature in °C
MCU → Laptop	LIGHT:DARK/BRIGHT	Room light state
MCU → Laptop	MOTION:<n>	Motion count inside current window
MCU → Laptop	AWAKE	Baby declared awake (≥4 motions)
MCU → Laptop	ALERT:GAS_DETECTED	Gas threshold crossed
MCU → Laptop	ACK:...	Command acknowledgement

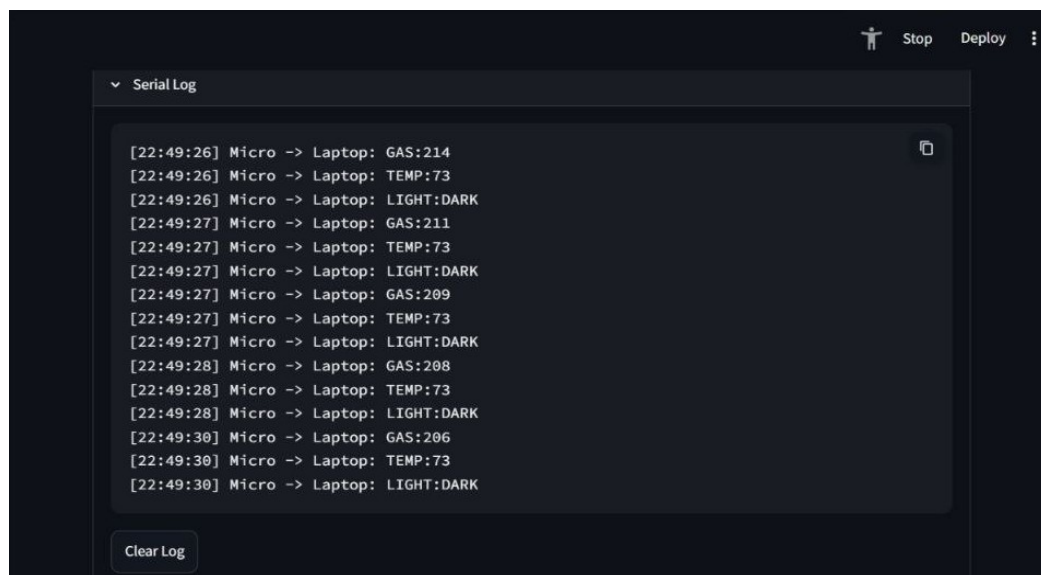


Figure 6 – Live Serial Log showing continuous GAS / TEMP / LIGHT telemetry

6. Machine Learning Pipeline:

6.1 Dataset

We used the publicly available **Donate-a-Cry Corpus** (457 labeled recordings). A pre-extracted feature version was obtained from Kaggle for rapid prototyping, while a matching feature-extraction routine was also implemented for live inference.

Class	Recordings	Percentage	System Response
Hungry	382	≈ 84 %	Display message + optional calming video
Discomfort	27	≈ 6 %	Display message (check diaper / position)
Tired	24	≈ 5 %	Continuous buzzer + urgent warning

Note: The strong class imbalance is a well-known challenge of this dataset. SMOTE / RandomOverSampler inside the cross-validation pipeline and careful macro-F1 evaluation were used to mitigate it.

6.2 Feature Set (25 acoustic features)

Feature Group	Examples
Amplitude / Energy	Amplitude_Envelope_Mean, RMS_Mean
Temporal	ZCR_Mean
Spectral	STFT_Mean, SC_Mean, SBAN_Mean, SCON_Mean, MelSpec
MFCC	MFCCs13Mean, MFCCs1 ... MFCCs13, MFCCs20
Delta MFCC	delMFCCs13, del2MFCCs13

6.3 Training Strategy

- 80 / 20 stratified train-test split (random_state = 42)
- 5-fold Stratified Cross-Validation inside the training set
- Pipeline: Imputer → Scaler → (optional) SMOTE → Classifier
- Models evaluated: Logistic Regression, SVM, Random Forest, Extra Trees, HistGradientBoosting, KNN, VotingClassifier
- Selection criterion: highest CV macro-F1 (balanced performance across the three classes)
- Final artifact saved as a single joblib file containing pipeline, feature names, class names

6.4 Live Inference Path

When a new audio file (or microphone buffer) arrives, the same feature-extraction recipe is applied, the resulting vector is fed to the loaded pipeline, and the predicted label plus confidence are returned. The GUI then issues the corresponding serial command and Telegram notification.

7. System Workflow (End-to-End):

7.1 Cry Detection & Classification

1. Laptop microphone (or uploaded file) captures audio.
2. Optional spectral noise reduction + voice-activity detection.
3. Fixed-length (5 s) feature vector is extracted.
4. Trained model returns one of {hungry, tired, discomfort} + confidence.
5. Streamlit updates status cards and log.
6. Serial command CRY:<label> is sent to the microcontroller.
7. Telegram message is dispatched to the caregiver.

7.2 Hardware Reaction

- Servo immediately begins a continuous rocking cycle ($0^\circ \rightarrow 90^\circ \rightarrow 180^\circ$).
- LCD shows “Baby Crying! Reason: <label>”.
- If the label is tired, the buzzer stays on continuously.
- Room lamp turns on if the room is dark.

7.3 Environmental & Safety Loops

- PIR motion window (8 s) decides whether the baby is awake.
 - Temperature band drives RGB LED colour and fan PWM.
 - Gas sensor continuously compared against threshold 400; on breach the system enters the highest-priority SAFETY ALERT state (buzzer + full-screen GUI + Telegram).
-

8. Implementation Details & Screenshots

8.1 Gas Safety Alert (Highest Priority)

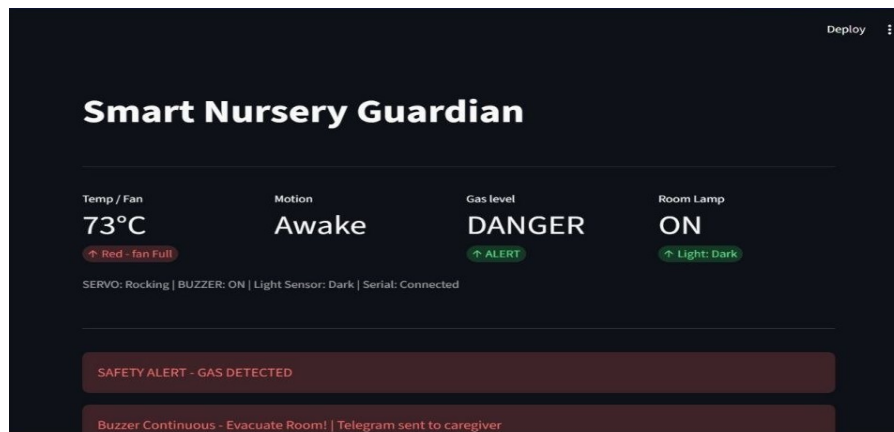


Figure 7 – GUI in SAFETY ALERT state: Gas = DANGER, Servo rocking, Buzzer ON, Telegram sent

8.2 Temperature / Calm State

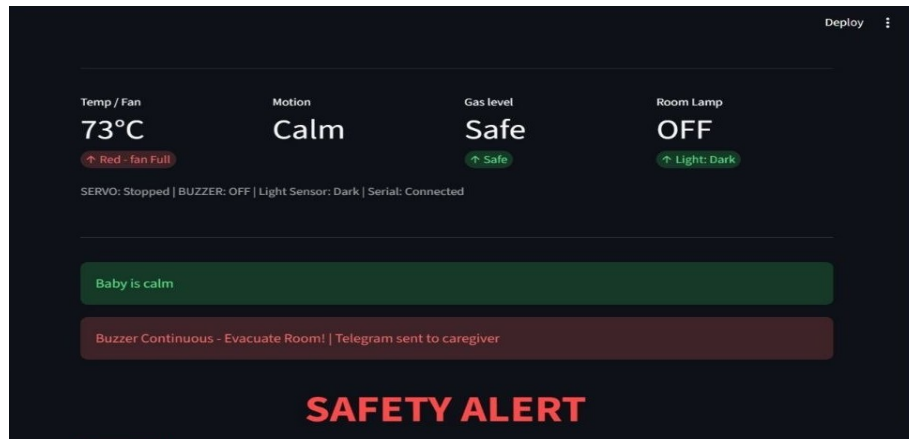


Figure 8 – High temperature (73 °C in test) with Red-fan-Full indicator; baby calm

8.3 Telegram Bot Integration

A dedicated Telegram bot (@SMART_NURSERY_GUARDIAN_bot) receives every cry classification and every gas alert, giving the caregiver immediate mobile visibility even when away from the laptop. When gas is detected (or a cry is classified), the system sends a Telegram message to a pre-configured chat so the caregiver gets a phone alert even from another room. Implementation: a bot created through @BotFather provides the API token; the chat ID is read from the first getUpdates response; alerts are sent with python-telegram-bot — free, serverless, and far more reliable than browser-automation alternatives. Cry alerts are deduplicated per label and gas alerts are rate-limited to one message every 30 seconds to avoid notification spam.



Figure 9 – Telegram bot profile

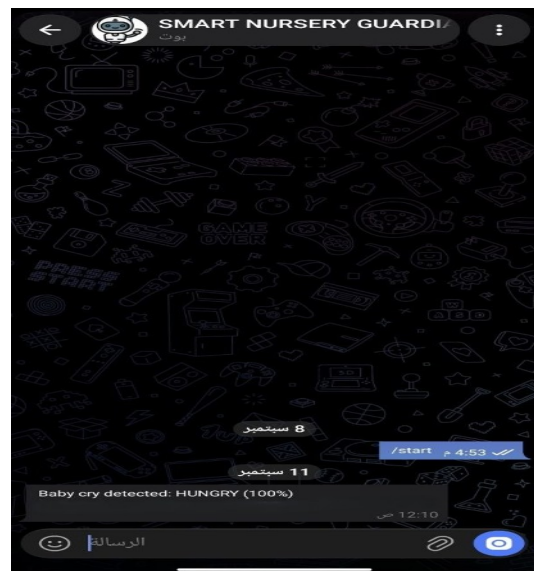


Figure 10 – Real-time cry notification: “Baby cry detected: HUNGRY (100%)”

9. Testing & Results:

9.1 Functional Test Matrix

Test Case	Input / Stimulus	Expected Outcome	Result
Hungry cry (upload)	Audio file labelled hungry	GUI "hungry", CRY:hungry, Telegram	Pass
Tired cry (mock)	Sidebar "Try Tired"	Continuous buzzer + LCD "Tired"	Pass
Discomfort cry	Sidebar "Try Discomfort"	LCD message + rock, no buzzer	Pass
Gas threshold	Sensor > 400 or mock button	Full SAFETY ALERT + Telegram	Pass
Motion → Awake	≥ 4 PIR edges in 8 s	isBabyAwake, LCD "Awake"	Pass
Dark + Awake → Lamp	LDR dark + awake/cry	Room LED ON	Pass
Temperature bands	Thermistor readings	Correct RGB + fan PWM	Pass
Reset command	RESET / Clear button	All actuators return to idle	Pass
Serial telemetry	Continuous monitoring	GAS/TEMP/LIGHT every ≈1 s	Pass

9.2 Machine-Learning Evaluation Notes

Because of the severe class imbalance, overall accuracy alone is misleading. The final model was selected on the basis of macro-averaged F1 and balanced accuracy obtained from stratified 5-fold cross-validation. The saved artifact (smart_nursery_guardian_3class_best_model.pkl) contains the complete scikit-learn / imbalanced-learn pipeline and can be loaded once at GUI start-up for all subsequent inferences.

10. Challenges & Solutions:

Challenge	Solution Adopted
Severe class imbalance (hungry ≈ 84 %)	SMOTE / RandomOverSampler inside CV pipeline; selection by macro-F1 instead of accuracy
Feature schema mismatch risk	Strict feature-name validation before prediction; same extraction code used for training & live
Serial flooding / latency	1-second telemetry throttle; non-blocking Arduino loop
False awake detections	8-second sliding window + ≥ 4 motion edges required
Telegram spam	Per-label deduplication + 30-second rate limit on gas alerts
Streamlit thread-safety	Best-effort session-state updates from background mic thread; manual upload path remains fully synchronous

11. Conclusion:

Smart Nursery Guardian successfully demonstrates a complete, end-to-end intelligent infant-care system that combines classical machine learning, real-time embedded control, a modern web dashboard, and remote mobile notifications.

The three-class cry classifier, trained on the Donate-a-Cry corpus and carefully regularised against class imbalance, provides actionable guidance (hungry / tired / discomfort). The microcontroller reacts within milliseconds by rocking the crib, illuminating the room when needed, and raising an unmistakable safety alarm when gas is detected. Caregivers receive immediate Telegram messages and can monitor the entire system from a single Streamlit interface.

By splitting responsibilities — high-level AI and GUI on the laptop, low-level sensing and actuation on the microcontroller — the architecture mirrors professional IoT practice and remains maintainable and extensible. The project therefore fulfils both its technical and its human-centred goals: reducing parental uncertainty and improving the safety and comfort of the infant.

12. Future Work:

- Expand the cry taxonomy (pain, burping, environmental noise) with additional labelled data.
 - Replace classical models with a lightweight spectrogram CNN or self-supervised audio encoder once more balanced data become available.
 - Add on-device (edge) inference using TensorFlow Lite Micro or Edge Impulse for fully offline operation.
 - Integrate a camera for visual confirmation of sleep posture and face occlusion.
 - Multi-room / multi-crib support via MQTT or a local Wi-Fi mesh.
 - Long-term data logging and parental insights dashboard (sleep patterns, cry frequency trends).
 - Formal clinical validation study with paediatricians.
-

13. Resources & References:

13.1 Project Repositories & Media

- GitHub Repository: <https://github.com/Mohamed-Teba/Smart-Nursery-Guardian-System>

13.2 Datasets

- Infant Cry Audio Corpus – Kaggle: <https://www.kaggle.com/datasets/warcoder/infant-cry-audio-corpus>
- Pre-extracted Features – Kaggle:
<https://www.kaggle.com/datasets/bhoomikavalani/donateacrycorpusfeaturesdataset>

13.3 Libraries & Frameworks

- Python 3.12, scikit-learn, imbalanced-learn, librosa, joblib
- Streamlit (GUI), pyserial, python-telegram-bot
- Arduino / AVR C++ (LiquidCrystal_I2C, Servo)
- Altium Designer (PCB)

13.4 Academic & Technical References

- G. Veres et al., “Donate-a-Cry Corpus” – public infant cry dataset used for training.
- Published studies on MFCC-based infant cry classification reporting >90 % accuracy after proper balancing and augmentation.
- scikit-learn documentation – Pipeline, StratifiedKFold, metrics.
- librosa documentation – feature extraction (MFCC, spectral, RMS, ZCR).
- Streamlit documentation – session state, caching, file upload.
- Arduino Reference – Servo, analogRead, Serial protocol design patterns.

Team 6 — Mohamed Ibrahim (7131E), Filopateer Hani (7129E), Tharaa Mohamed (7005E)
Smart Nursery Guardian — Final Project Documentation